

```

"""
universal_tts.py

Plug-and-play TTS layer: swappable free/open-source
backends behind
one interface, a tag system for emotion/pauses/non-
speech sounds, and
voice profile management.

Backends (pick based on what you need):
- Bark      : best tag/non-speech support (laughs,
sighs, pauses).
                pip install git+https://github.com/suno-
ai/bark.git
- XTTS v2   : best raw cloning quality from ~6s
reference audio.
                NON-COMMERCIAL license (CPML) – personal
use only.
                pip install TTS
- Kokoro    : lightest/fastest, runs on CPU, Apache-2.0
(commmercial OK).
                No cloning – good fallback on current
hardware or once
                you're selling copies of the bot.
                pip install kokoro
"""

import os
import re
import json
import numpy as np
from dataclasses import dataclass, field
from abc import ABC, abstractmethod
from typing import Optional, List

# -----
-----

```

```

# VOICE PROFILE
# -----
-----

@dataclass
class VoiceProfile:
    """
    A voice you can generate speech in.
    reference_audio_path is only
    needed for cloning backends (XTTS); Bark/Kokoro
    use built-in
    preset voices instead.

    consent_confirmed exists on purpose: cloning
    backends refuse to
    run without it. Set it True only for your own
    voice, a
    public-domain recording, or someone who's
    explicitly agreed to
    be cloned.
    """
    name: str
    reference_audio_path: Optional[str] = None
    preset_id: Optional[str] = None      # for
Bark/Kokoro built-in voices
    language: str = "en"
    consent_confirmed: bool = False
    description: str = ""

    def validate_for_cloning(self):
        if self.reference_audio_path and not
self.consent_confirmed:
            raise PermissionError(
                f"VoiceProfile '{self.name}' has
reference audio but "
                "consent_confirmed=False. Set it True
only if this is "
                "your own voice or you have explicit
permission to use it."
            )

```

```

class VoiceLibrary:
    """Upload/register/manage voice profiles,
    persisted to disk."""

    def __init__(self, storage_dir="voices"):
        self.storage_dir = storage_dir
        os.makedirs(storage_dir, exist_ok=True)
        self.profiles = {}
        self._load_index()

    def _index_path(self):
        return os.path.join(self.storage_dir,
                             "index.json")

    def _load_index(self):
        path = self._index_path()
        if os.path.exists(path):
            with open(path) as f:
                raw = json.load(f)
                self.profiles = {k: VoiceProfile(**v) for
                                  k, v in raw.items()}

    def _save_index(self):
        with open(self._index_path(), "w") as f:
            json.dump({k: vars(v) for k, v in
                       self.profiles.items()}, f, indent=2)

    def upload_voice(self, name, audio_file_path,
                     language="en",
                     consent_confirmed=False,
                     description=""):
        """Register a reference clip. Copies it into
        managed storage."""
        import shutil
        dest = os.path.join(self.storage_dir, f"
{name}.wav")
        shutil.copy(audio_file_path, dest)

        profile = VoiceProfile(
            name=name,
            reference_audio_path=dest,
            language=language,

```

```

        consent_confirmed=consent_confirmed,
        description=description,
    )
    self.profiles[name] = profile
    self._save_index()
    return profile

    def get(self, name) -> VoiceProfile:
        return self.profiles[name]

# -----
# TAG SYSTEM – emotion, pauses, non-speech sounds
# -----

@dataclass
class Segment:
    text: str
    tags: List[str] = field(default_factory=list)
    pause_after_ms: int = 0

# Tags usable directly in generated text:
# [happy] [sad] [angry] [jealous] [tired] [excited]
# [whisper] [horny] [annoyed]
# [laughs] [sighs] [gasps] [clears_throat]
# [chuckles]
# [pause:300] -> 300ms silence after the preceding
# chunk
TAG_PATTERN = re.compile(r"\([([a-z_]+)(?::(\d+))?\)")

class TagProcessor:
    """Parses inline tags out of generated text into
    structured segments."""

    EMOTION_TAGS = {"happy", "sad", "angry",
                    "jealous", "tired",
                    "excited", "whisper", "serious",
                    "horny", "annoyed"}

```

```

        SOUND_TAGS = {"laughs", "sighs", "gasps",
"clears_throat", "chuckles"}

    def parse(self, raw_text: str) -> List[Segment]:
        segments = []
        active_tags = []
        pos = 0

        for match in TAG_PATTERN.finditer(raw_text):
            chunk =
raw_text[pos:match.start()].strip()
            if chunk:
                segments.append(Segment(text=chunk,
tags=list(active_tags)))

                tag_name, pause_ms = match.group(1),
match.group(2)

                if tag_name == "pause" and pause_ms:
                    if segments:
                        segments[-1].pause_after_ms =
int(pause_ms)
                    elif tag_name in self.SOUND_TAGS:
                        segments.append(Segment(text=f"
[{tag_name}]", tags=["_sound_"]))
                    elif tag_name in self.EMOTION_TAGS:
                        active_tags = [tag_name]

                pos = match.end()

            tail = raw_text[pos:].strip()
            if tail:
                segments.append(Segment(text=tail,
tags=list(active_tags)))

        return segments

    def to_bark_format(self, segments: List[Segment])
-> str:
        """Bark understands bracket tags natively –
reassemble directly."""
        out = []

```

```

        for seg in segments:
            if seg.tags == ["_sound_"]:
                out.append(seg.text)
                continue
            piece = f"[whispers] {seg.text}" if
"whisper" in seg.tags else seg.text
            out.append(piece)
            if seg.pause_after_ms:
                out.append("..." if seg.pause_after_ms
< 500 else "... ...")
            return " ".join(out)

    def to_plain_with_pauses(self, segments:
List[Segment]):
        """
        For backends with no native tag support (XTTS,
Kokoro): strip
        tags to clean text, return pause points so the
engine can
        splice real silence in during post-processing.
        """
        clean_parts = []
        pause_points = []
        running_text = ""

        for seg in segments:
            if seg.tags == ["_sound_"]:
                continue
            running_text += seg.text + " "
            if seg.pause_after_ms:

pause_points.append((len(running_text),
seg.pause_after_ms))

        return running_text.strip(), pause_points

def mood_to_tagged_text(text: str, mood: str,
mood_level: int) -> str:
    """
    Bridges your existing mood system straight into
TTS tags – call

```

```
        this on the bot's reply text before passing it to
        TTSEngine.speak().
    """
```

```
        mood_map = {
            "happy": "happy", "excited": "excited",
            "annoyed": "annoyed",
            "angry": "angry", "sad": "sad", "jealous":
            "jealous",
            "tired": "tired", "stressed": "annoyed",
            "horny": "horny",
            "neutral": None,
        }
        tag = mood_map.get(mood)
        if not tag:
            return text
        prefix = f"[{tag}] " if mood_level >= 4 else ""
        return prefix + text
```

```
# -----
# -----
# BACKENDS
# -----
# -----
```

```
class TTSEBackend(ABC):
    name = "base"
    supports_native_tags = False
    supports_cloning = False

    @abstractmethod
    def synthesize(self, text: str, voice:
Optional[VoiceProfile]) -> np.ndarray:
        ...
```

```
class BarkBackend(TTSEBackend):
    name = "bark"
    supports_native_tags = True
    supports_cloning = False    # preset-based, not
true cloning
```

```

def __init__(self):
    from bark import generate_audio,
preload_models
    preload_models()
    self._generate_audio = generate_audio

    def synthesize(self, text, voice=None):
        history_prompt = voice.preset_id if voice else
None
        return self._generate_audio(text,
history_prompt=history_prompt)

class XTTSBackend(TTSBackend):
    name = "xtts"
    supports_native_tags = False
    supports_cloning = True

    def __init__(self):
        from TTS.api import TTS
        self.model =
TTS("tts_models/multilingual/multi-dataset/xtts_v2")

    def synthesize(self, text, voice=None):
        if voice:
            voice.validate_for_cloning()
            return self.model.tts(
                text=text,
                speaker_wav=voice.reference_audio_path if
voice else None,
                language=voice.language if voice else
"en",
            )

class KokoroBackend(TTSBackend):
    name = "kokoro"
    supports_native_tags = False
    supports_cloning = False

    def __init__(self):
        from kokoro import KPipeline

```

```

        self.pipeline = KPipeline(lang_code="a")

    def synthesize(self, text, voice=None):
        voice_id = voice.preset_id if voice else
        "af_heart"
        generator = self.pipeline(text,
        voice=voice_id)
        audio_chunks = [audio for _, _, audio in
        generator]
        return np.concatenate(audio_chunks)

# -----
# UNIFIED ENGINE
# -----

class TTSEngine:
    """
    engine = TTSEngine(backend="bark")
    engine.voices.upload_voice("me", "my_sample.wav",
    consent_confirmed=True)
    audio = engine.speak("[happy] omg baby I missed
    you [laughs] fr fr")
    engine.save(audio, "reply.wav")
    """

    def __init__(self, backend: str = "bark",
    voices_dir="voices"):
        self.tag_processor = TagProcessor()
        self.voices = VoiceLibrary(voices_dir)
        self.backend = self._load_backend(backend)

    def _load_backend(self, name) -> TTSBackend:
        return {"bark": BarkBackend, "xtts":
        XTTSBackend, "kokoro": KokoroBackend}[name]()

    def speak(self, tagged_text: str, voice_name:
    Optional[str] = None) -> np.ndarray:
        voice = self.voices.get(voice_name) if
        voice_name else None

```

```

        segments =
self.tag_processor.parse(tagged_text)

        if self.backend.supports_native_tags:
            final_text =
self.tag_processor.to_bark_format(segments)
            return self.backend.synthesize(final_text,
voice)

        clean_text, pause_points =
self.tag_processor.to_plain_with_pauses(segments)
        audio = self.backend.synthesize(clean_text,
voice)
        return self._insert_pauses(audio,
pause_points, clean_text)

    def _insert_pauses(self, audio, pause_points,
text, sample_rate=24000):
        if not pause_points:
            return audio
        chars_per_sample = len(text) / max(len(audio),
1)
        result, last_cut = [], 0
        for char_idx, ms in pause_points:
            sample_idx = min(int(char_idx /
chars_per_sample) if chars_per_sample else 0,
len(audio))
            result.append(audio[last_cut:sample_idx])
            result.append(np.zeros(int(sample_rate *
ms / 1000)))
            last_cut = sample_idx
            result.append(audio[last_cut:])
        return np.concatenate(result)

    def save(self, audio: np.ndarray, path: str,
sample_rate=24000):
        from scipy.io.wavfile import write
        write(path, sample_rate, audio)

# -----
-----

```

```
# INTEGRATION NOTE
# -----
-----
"""
Wire this into the ApprovalQueue from
dynamic_prompt_engine.py the
same way as text drafts: generate the voice note,
attach it to the
DraftMessage as pending, and only hand it to
whatsapp.js for sending
after you approve it in the dashboard. Same human-in-
the-loop gate,
now for audio too.
"""
```